

Detection Engineering & Resilience Lab

SOC Lab Report

Shadi Jahangiri

Date: 25/09/2026

Table of Contents

Contents

Table of Contents..... 2

1. Project Overview..... 3

2. Architecture 3

 2.1 Forwarder Configuration (outputs.conf) 3

 2.2 Inputs Added (inputs.conf, local) 4

 2.3 Confirmed Forwarding Status 4

3. Brute-Force Detection (T1110) 4

 3.1 Initial Investigation..... 4

 3.2 Field Discovery (Debugging Finding) 5

 3.3 Detection Tuning..... 5

 3.4 Sigma Rule (Vendor-Neutral Equivalent) 6

 3.5 MITRE ATT&CK Mapping..... 6

4. Detection Health Monitor..... 6

 4.1 Design — Two Independent Checks 7

 4.2 Heartbeat Generation..... 7

 4.3 Test Sequence and Results 8

 4.4 Tier 2 Field-Presence Check 9

 4.5 monitor.py — Offline Health Report Tool..... 9

5. Limitations..... 11

6. Conclusion..... 11

1. Project Overview

Environment: WS01 (DESKTOP-QJ4S25N) → Splunk Universal Forwarder → Splunk (LAPTOP-SHGCEECC)

This project is a small SOC lab built around one main question: can you tell when a security detection has quietly stopped working properly, not because the rule itself is wrong, but because the logs it relies on have stopped arriving or are missing the fields it needs?

A detection is only as good as the data feeding it. If a SOC sees zero alerts, that could mean nothing happened or it could mean the logging pipeline is broken. Those two situations look exactly the same on a dashboard, but they need completely different responses. This project first builds a brute-force detection the normal way (investigate, detect, tune, map to ATT&CK), then builds a smaller second system on top of it whose only job is to check whether that detection can actually still see what it needs to see.

2. Architecture

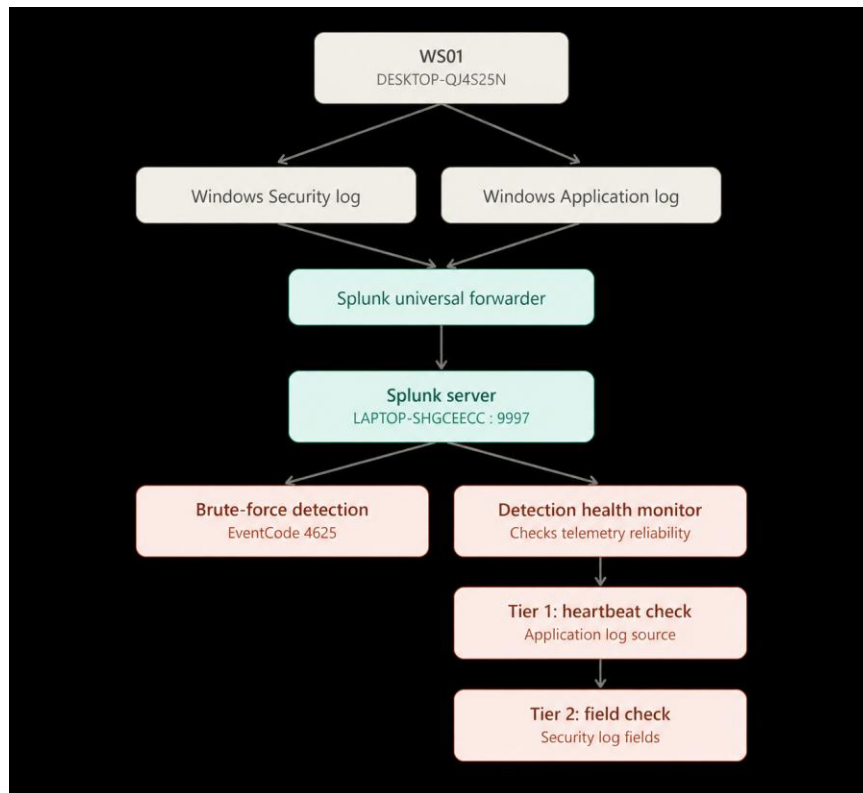


Figure 1. SOC Lab Architecture and Detection Health Monitor

2.1 Forwarder Configuration (outputs.conf)

```
[tcpout]
defaultGroup = default-autolb-group
```

```
[tcpout:default-autolb-group]
server = 192.168.0.27:9997
```

2.2 Inputs Added (inputs.conf, local)

```
[WinEventLog://Application]
disabled = 0
```

(Security and Sysmon inputs were already present from initial setup.)

2.3 Confirmed Forwarding Status

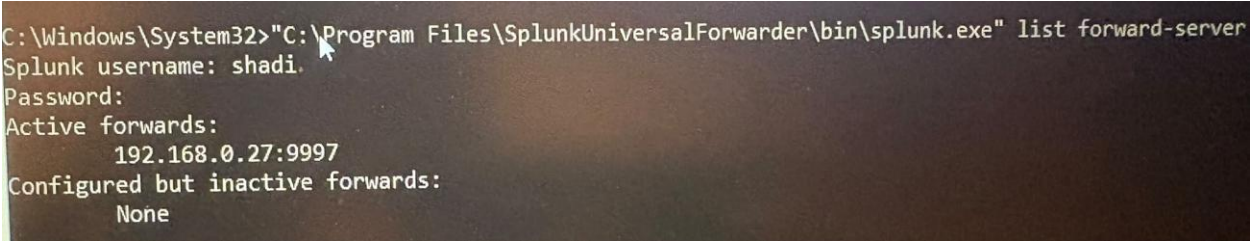


Figure 2. Splunk Universal Forwarder connected successfully to the Splunk server on port 9997.

3. Brute-Force Detection (T1110)

3.1 Initial Investigation

Baseline search to confirm Windows Security telemetry was flowing before building any detection:

```
index=* source="WinEventLog:Security"
| stats count as security_events latest(_time) as last_seen
| eval last_seen=strftime(last_seen,"%Y-%m-%d %H:%M:%S")
```

Result: 228 Windows Security events received over a 4-hour window — this served as the healthy-state baseline before any deliberate telemetry disruption was introduced later in the project.

Time	Event
20/09/2026 17:30:56.243	09/20/2026 05:30:56.243 PM LogName=Security EventCode=4625 EventType=0 ComputerName=DESKTOP-QJ4S25N Show all 61 lines host = DESKTOP-QJ4S25N : source = WinEventLog:Security : sourcetype = WinEventLog:Security
20/09/2026 17:30:52.512	09/20/2026 05:30:52.512 PM LogName=Security EventCode=4625 EventType=0 ComputerName=DESKTOP-QJ4S25N Show all 61 lines host = DESKTOP-QJ4S25N : source = WinEventLog:Security : sourcetype = WinEventLog:Security
20/09/2026 17:30:48.086	09/20/2026 05:30:48.086 PM LogName=Security EventCode=4625 EventType=0 ComputerName=DESKTOP-QJ4S25N Show all 61 lines host = DESKTOP-QJ4S25N : source = WinEventLog:Security : sourcetype = WinEventLog:Security

Figure 3. Windows Event ID 4625 showing a failed logon for the labuser account.

Investigating failed logons specifically:

```
index=* source="WinEventLog:Security" "4625"
```

followed by inspecting caller process, logon type, and failure reason for each event to understand what a real failed-logon event looks like on this host before building a detection around it.

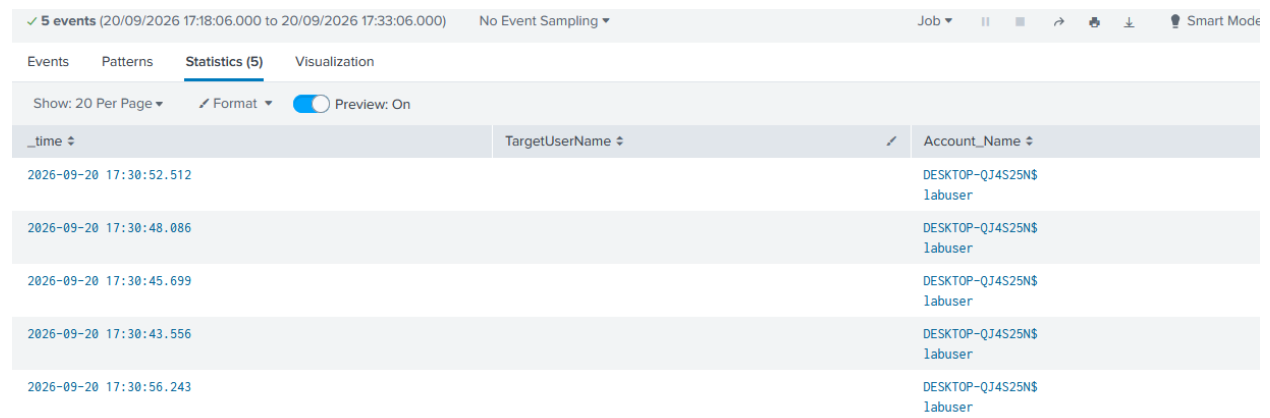
3.2 Field Discovery (Debugging Finding)

The standard Sigma/Splunk field TargetUserName did not extract usefully in this environment. Instead, the account that failed to log on appeared in the multi-value field Account_Name (index 1), alongside the machine account at index 0:

```
index=* source="WinEventLog:Security" "4625"
| table _time TargetUserName Account_Name

index=* source="WinEventLog:Security" "4625"
| eval failed_user=mvindex(Account_Name,1)
| stats count by failed_user
```

Because of this, every later query - the detection itself, the tuning, and the health monitor - uses `mindex(Account_Name,1)` instead of the field name.



_time	TargetUserName	Account_Name
2026-09-20 17:30:52.512	labuser	DESKTOP-QJ4S25N\$labuser
2026-09-20 17:30:48.086	labuser	DESKTOP-QJ4S25N\$labuser
2026-09-20 17:30:45.699	labuser	DESKTOP-QJ4S25N\$labuser
2026-09-20 17:30:43.556	labuser	DESKTOP-QJ4S25N\$labuser
2026-09-20 17:30:56.243	labuser	DESKTOP-QJ4S25N\$labuser

Figure 4. Comparison of TargetUserName and Account_Name fields in failed-logon events.

3.3 Detection Tuning

The first version of the rule just counted 5 failed logins total, with no time limit. That's too loose - someone mistyping their password 5 times across a whole day isn't an attack, it's just a bad memory. So the question became: what time window actually separates a typo from an attack?

I settled on: 5+ failed logins for the same account within a 1-minute window, which is a different claim than "5 failures at some point":

```
index=* source="WinEventLog:Security" "4625"
| eval failed_user=mindex(Account_Name,1)
```

```
| bin _time span=1m
| stats count values(host) as host values(Source_Network_Address) as src_ip
  by _time failed_user
| where count>=5
```

This was saved as a Splunk alert.

_time ↕	failed_user ↕	count ↕	host ↕	src_ip ↕
2026-09-20 17:30:00	labuser	5	DESKTOP-QJ4S25N	127.0.0.1

Figure 5. Tuned brute-force detection identifying five failed logons within a one-minute window.

3.4 Sigma Rule (Vendor-Neutral Equivalent)

```
title: Windows Failed Logon
id: 11111111-1111-1111-1111-111111111111
status: experimental
logsource:
  product: windows
  service: security
detection:
  selection:
    EventID: 4625
  condition: selection
---
title: Repeated Windows Failed Logons
correlation:
  type: event_count
  rules:
    - Windows Failed Logon
  group-by:
    - TargetUserName
  timespan: 1m
  condition:
    gte: 5
```

3.5 MITRE ATT&CK Mapping

Technique	Tactic	Justification
T1110 - Brute Force	Credential Access	Detection identifies repeated failed authentication attempts against a single account within a short window, consistent with password-guessing behaviour.

4. Detection Health Monitor

This part of the project is a small tool that checks whether the brute-force detection can actually be trusted right now, instead of just assuming zero alerts means zero problems.

4.1 Design — Two Independent Checks

Tier 1 - Pipeline health (heartbeat). Is data reaching Splunk at all? A uniquely-tagged test event gets written to the Windows Application log (deliberately not Sysmon or Security, so this check can't be broken by the same misconfiguration that would break the detection it's supposed to be validating), then Splunk is queried for that exact ID.

Tier 2 - Telemetry/schema health. Even if the pipeline is alive, does the actual 4625 event still have the fields the detection needs? This checks that Account_Name and Source_Network_Address aren't empty.

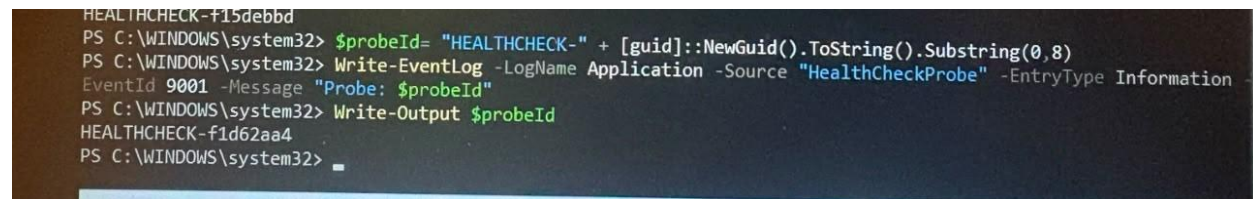
Keeping these two checks separate means the tool can tell apart three different failure states instead of just pass/fail:

- Pipeline down → detection UNRELIABLE, reason: heartbeat missing
- Pipeline up, required field missing → detection UNRELIABLE, reason: schema drift
- Pipeline up, fields present → detection RELIABLE

4.2 Heartbeat Generation

```
New-EventLog -LogName Application -Source "HealthCheckProbe" # one-time setup
```

```
$probeId = "HEALTHCHECK-" + [guid]::NewGuid().ToString().Substring(0,8)
Write-EventLog -LogName Application -Source "HealthCheckProbe" `
  -EntryType Information -EventId 9001 -Message "Probe: $probeId"
Write-Output $probeId
```



```
HEALTHCHECK-f15debdb
PS C:\WINDOWS\system32> $probeId= "HEALTHCHECK-" + [guid]::NewGuid().ToString().Substring(0,8)
PS C:\WINDOWS\system32> Write-EventLog -LogName Application -Source "HealthCheckProbe" -EntryType Information -
EventId 9001 -Message "Probe: $probeId"
PS C:\WINDOWS\system32> Write-Output $probeId
HEALTHCHECK-f1d62aa4
PS C:\WINDOWS\system32>
```

Figure 6. Synthetic heartbeat verified locally in the Windows Application log.

Verified locally with:

```
Get-WinEvent -FilterHashtable @{LogName='Application'; ProviderName='HealthCheckProbe'; Id=9001} `
  -MaxEvents 5 | Format-List TimeCreated, Id, ProviderName, Message
```

and in Splunk with: `index=* source="WinEventLog:Application" EventCode=9001`

```
> 20/09/2026 09/20/2026 10:09:13.797 PM
22:09:13.797 LogName=Application
EventCode=9001
EventType=4
ComputerName=DESKTOP-QJ4S25N
SourceName=HealthCheckProbe
Type=Information
RecordNumber=1003
Keywords=Classic
TaskCategory=1
OpCode=Info
Message=Probe: HEALTHCHECK-f1d62aa4
Collapse
host = DESKTOP-QJ4S25N | source = WinEventLog:Application | sourcetype = WinEventLog:Application
```

Figure 7. Synthetic heartbeat received successfully in Splunk.

4.3 Test Sequence and Results

Stage	Action	Result
Healthy	Heartbeat generated, forwarder running	HEALTHCHECK-f1d62aa4 received in Splunk within seconds. age_minutes ≈ 0
Pipeline degraded	Stop-Service SplunkForwarder, new heartbeat generated	New probe never arrived in Splunk (confirmed via netstat and Test-NetConnection -> TcpTestSucceeded: False)
Recovered	Forwarder restarted, new heartbeat generated	New probe arrived; telemetry age returned to ≈ 0 minutes
Schema degraded	Controlled simulation of a missing required field in a 4625 event	Pipeline reported healthy, but Tier 2 flagged the missing field

Telemetry-age query used to quantify staleness during the pipeline-down test:

```
index=* source="WinEventLog:Security"
| stats latest(_time) as last_seen
| eval age_minutes=round((now()-last_seen)/60,1)
| eval last_seen=strftime(last_seen,"%Y-%m-%d %H:%M:%S")
```

Result during the degraded window: 6.2 minutes stale, compared to ≈ 0.2 minutes in the healthy baseline - so there's an actual number behind "broken" rather than just a claim.

✓ 172 events (20/09/2026 19:54:05.000 to 20/09/2026 20:09:05.000)		No Event Sampling ▼	Job ▼	II	■	↶	↷	⬇	Smart Mode ▼
Events	Patterns	Statistics (1)	Visualization						
Show: 20 Per Page ▼		Format ▼	Preview: On						
last_seen ↕					age_minutes ↕				
2026-09-20 20:02:53					6.2				

Figure 8. Security telemetry staleness during the pipeline-degraded test.

4.4 Tier 2 Field-Presence Check

```
index=* source="WinEventLog:Security" EventCode=4625
| eval failed_user=mindex(Account_Name,1)
| eval account_ok=if(isnotnull(failed_user) AND failed_user!="", "YES", "NO")
| eval source_ip_ok=if(isnotnull(Source_Network_Address) AND Source_Network_Address!="", "YES", "NO")
| table _time failed_user Source_Network_Address account_ok source_ip_ok
```

The healthy result confirmed that both required fields were present in the 4625 failed-logon event, with Account_Name successfully identifying labuser and Source_Network_Address showing 127.0.0.1. Therefore, Tier 2 considered the detection telemetry healthy.

✓ 2 events (20/09/2026 22:07:42.000 to 20/09/2026 22:22:42.000)		No Event Sampling ▼	Job ▼	II	■	↶	↷	⬇	Smart Mode ▼
Events	Patterns	Statistics (2)	Visualization						
Show: 20 Per Page ▼		Format ▼	Preview: On						
_time ↕	failed_user ↕	Source_Network_Address ↕	account_ok ↕	source_ip_ok ↕					
2026-09-20 22:22:03.342	labuser	127.0.0.1	YES	YES					
2026-09-20 22:22:01.506	labuser	127.0.0.1	YES	YES					

Figure 9. Tier 2 field check confirming the required 4625 fields are present.

✓ 2 events (20/09/2026 22:10:02.000 to 20/09/2026 22:25:02.000)		No Event Sampling ▼	Job ▼	II	■	↶	↷	⬇	Smart Mode ▼
Events	Patterns	Statistics (2)	Visualization						
Show: 20 Per Page ▼		Format ▼	Preview: On						
_time ↕	failed_user ↕	Source_Network_Address ↕	account_ok ↕	source_ip_ok ↕					
2026-09-20 22:22:03.342	labuser	127.0.0.1	YES	NO					
2026-09-20 22:22:01.506	labuser	127.0.0.1	YES	NO					

Figure 10. Controlled Tier 2 degradation test showing the source IP field check failing.

4.5 monitor.py — Offline Health Report Tool

A small Python script reads a JSON sample - either exported live from Splunk, or a recorded state from one of the tests above - and prints a HEALTHY/DEGRADED/RELIABLE report without needing a live connection to Splunk. That means the health-check logic itself can be checked by anyone, even without access to my lab:

```
python monitor.py sample-data/healthy-real.json
python monitor.py sample-data/pipeline-degraded.json
python monitor.py sample-data/schema-degraded.json
python monitor.py sample-data/recovered.json
```

Verified output for all four states:

```

Sample: healthy-real.json
Pipeline: HEALTHY | Telemetry: HEALTHY | Detection: RELIABLE
Reason: Heartbeat arrived and the required 4625 fields are present

Sample: pipeline-degraded.json
Pipeline: DEGRADED | Telemetry: UNKNOWN | Detection: UNRELIABLE
Reason: Heartbeat missing - forwarding pipeline may be down

Sample: recovered.json
Pipeline: HEALTHY | Telemetry: HEALTHY | Detection: RELIABLE
Reason: Heartbeat arrived and the required 4625 fields are present

Sample: schema-degraded.json
Pipeline: HEALTHY | Telemetry: DEGRADED | Detection: UNRELIABLE
Reason: Missing field(s): Source_Network_Address

```

```

Microsoft Windows [Version 10.0.26200.9445]
(c) Microsoft Corporation. All rights reserved.

C:\Users\shadi\OneDrive\Documents\SOCproj\detection-health-monitor-final>python monitor.py sample-data\healthy-real.json

=====
DETECTION HEALTH REPORT
=====
Sample: healthy-real.json

Pipeline: HEALTHY
Brute-force telemetry: HEALTHY
Brute-force detection: RELIABLE
Reason: Heartbeat arrived and the required 4625 fields are present
=====

```

Figure 11. Python monitor HEALTHY

```

C:\Users\shadi\OneDrive\Documents\SOCproj\detection-health-monitor-final>python monitor.py sample-data\pipeline-degraded.json

=====
DETECTION HEALTH REPORT
=====
Sample: pipeline-degraded.json

Pipeline: DEGRADED
Brute-force telemetry: UNKNOWN
Brute-force detection: UNRELIABLE
Reason: Heartbeat missing - forwarding pipeline may be down
=====

```

Figure 12. Python monitor pipeline DEGRADED

```

C:\Users\shadi\OneDrive\Documents\SOCproj\detection-health-monitor-final>python monitor.py sample-data\schema-degraded.json

=====
DETECTION HEALTH REPORT
=====
Sample: schema-degraded.json

Pipeline: HEALTHY
Brute-force telemetry: DEGRADED
Brute-force detection: UNRELIABLE
Reason: Missing field(s): Source_Network_Address
=====

```

Figure 13. Python monitor schema DEGRADED

healthy-real.json is built from real Splunk exports captured during the live tests. pipeline-degraded.json records the state observed during the real forwarder-outage test, where the generated heartbeat did not reach Splunk.

5. Limitations

- This is a small, single-host lab (just WS01), not a production environment with real user noise or enterprise-scale log volume.
- schema-degraded.json is a controlled simulation of a missing field rather than a live schema-drift event captured from the environment.
- The Tier 2 check verifies whether two required fields are present and non-empty. It is not intended to be a full schema-validation system.
- TargetUserName not extracting properly is specific to this lab's setup - a different Splunk/Windows environment might not need the Account_Name/mvindex workaround.

6. Conclusion

This lab covers the normal detection-engineering steps - build a detection, investigate real events, tune out false positives, map to MITRE ATT&CK, write it as a Sigma rule - and then adds one more piece: it treats the detection didn't fire as an ambiguous signal rather than a clear one, and builds a small system to figure out which situation it actually is. The health monitor distinguishes three states: detection working, pipeline failure, and degraded detection telemetry. The tests demonstrate each condition and show how the monitor identifies why the detection can or cannot be trusted. The healthy state is grounded in real Splunk exports, while schema degradation is presented as a controlled simulation.